P2P File Sharing System

Lane Brinkman
North Central College,
Naperville, IL
Computer Science Department
lsbrinkman@noctrl.edu

Anthony Le
North Central College,
Naperville, IL
Computer Science Department
avle@noctrl.edu

Abstract—The goal of the P2P storage infrastructure is for you to be able to store a file on a remote system while preserving the confidentiality and integrity of the file.

I. Introduction

In this assignment you will perform a conceptual design of a P2P system that acts as a secure storage for files.

II. Method

A. System Architecture

The proposed Peer-to-Peer (P2P) storage infrastructure is designed to facilitate secure file storage and retrieval while ensuring confidentiality and integrity of the transmitted and stored data. The system architecture consists of two main components: client nodes running on Raspberry Pi 3 devices and a central server.

B. Implementation Details

- **File Encryption:** Prior to transmission, files are encrypted using the Advanced Encryption Standard (AES-128) encryption algorithm. This ensures that files remain confidential during transit and storage. The PyCryptodome library is utilized for AES encryption.
- **Hashing for Integrity:** To ensure the integrity of files during transmission and storage, each file is hashed using the Secure Hash Algorithm 256 (SHA2-256). This hash is then securely transmitted alongside the encrypted file and stored on the server. The SHA2-256 hashing algorithm is implemented using the PyCryptodome library.
- **Secure Storage:** Encrypted files and their corresponding SHA2-256 hashes are stored securely on the server. Access to stored files is restricted to authorized users only.
- **Authentication and Authorization:** Authentication mechanisms are implemented to verify the identity of client nodes before allowing file storage or retrieval operations. Authorization mechanisms ensure that only authorized users can access specific files.

C. Programing Environment

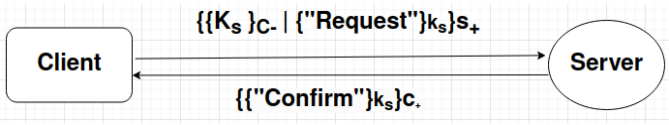
The implementation is carried out using Python3 programming language due to its simplicity and the availability of libraries like PyCryptodome. The development environment includes, Python, PyCryptodome library for cryptographic operations, Raspberry Pi 3 device for client implementation.

D. Experiment Setup

To evaluate the performance and security of the system, experiments will be conducted in a controlled environment using Raspberry Pi 3 devices as client nodes and a dedicated laptop as the server.

III. Result

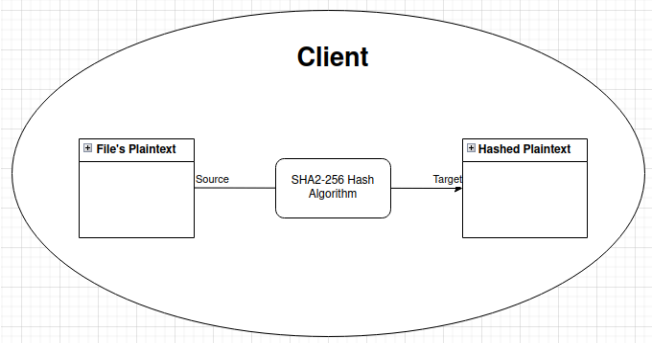
A. Initial session key creation



Here we assume that the Client's public key and the Server's public key are known to one another. The Client creates and then encrypts a session key (denoted as K_s) with its private key (denoted as C_-). The private key of the Client acts as an identifier for the Server, authenticating the Client as the true entity for the Server. A message is hidden that uses the session key ("Request"); the hidden message can only be unpackaged when the Server uses Client's public key to attain the session key, then and only then can the message be decrypted using the session key. Before these steps take place, the Server must use its private

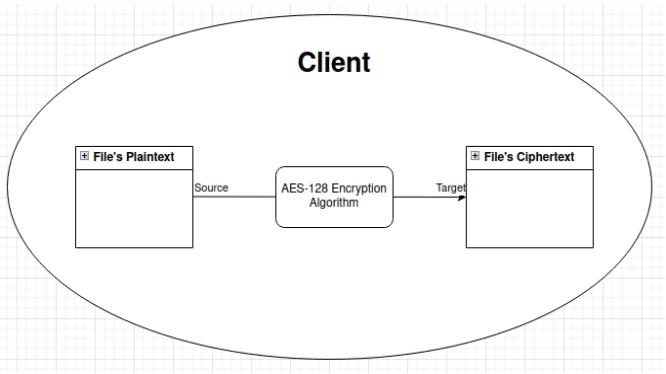
key to unpackage the first layer of encryption. This makes the request secure because the only entity that can unpackage the first layer of security is the Server because the Server is the only one with its private key. When the session key has been successfully shared with the Server, the Server is able to encrypt a message ("Confirm") with the shared session key that only the Client and the Server are aware of. The Server will add another layer of encryption with the Client's public key, to where the Client is the only one able to unpackage it using the Client's private key. This design allows the Client and the Server to communicate with one another securely, ensuring authentication between the two over a network connection.

B. Hashing Plaintext for Confidentiality and for Later File Integrity Check

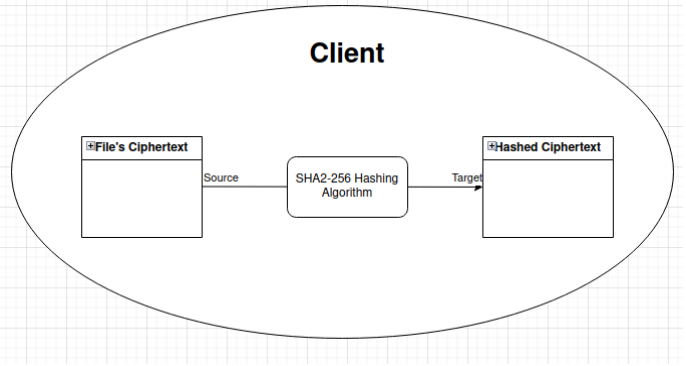


Here we see that hashing plaintext with SHA2-256 serves two crucial purposes when sending files to a server and storing them. The hash plaintext with SHA2-256 allows for an indirect way of confidentiality due to hash digest that when intercepted by unauthorized entities, they would not be able to reconstruct the original plaintext from the hash. After the file reaches the server, storing the hashed value of the file provides a means to verify its integrity later on. Since SHA2-256 produces a unique and fixed-size hash digest for each unique input, any alteration to the file content, no matter how small, will result in a different hash value. In summary, hashing plaintext with SHA2-256 before transmission ensures confidentiality during transit by obscuring the actual content of the file. Additionally, storing the hash value on the server enables later integrity checks, allowing for verification of file integrity and ensuring that the stored files remain unchanged and trustworthy over time.

C. Encrypting Plaintext for Confidentiality of File Before Being Transmitted



Here we see that we encrypt the file with the AES-128 encryption algorithm. Encrypting plaintext for confidentiality before transmission is crucial for ensuring the security of files stored on a server for several reasons. It helps protect against unauthorized access,



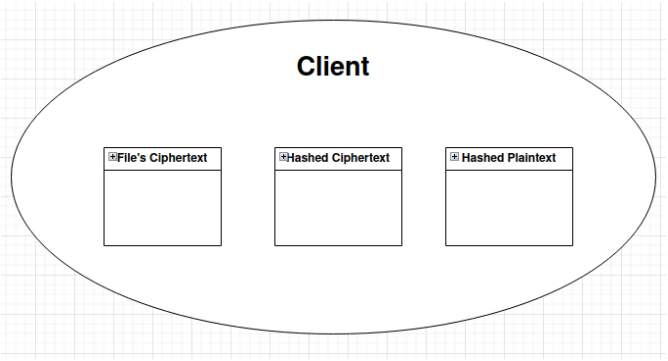
data tampering, insider threats, and ensures compliance with data privacy regulations, ultimately enhancing trust and reputation.

D. Hashing File's Ciphertext for File Integrity Check on Server

Server must not be able to decrypt the original encrypted file or see what is in the encrypted file, but in order to do a file integrity check we must see what is in the file. Since the server cannot see the file's plaintext at all, we must work around this functionality to where the server can do a file integrity check without seeing what is in the file. In order to accomplish this we must take a copy of the file's encrypted plaintext (ciphertext) and hash it using the SHA2-256 hashing algorithm. Since this hashed ciphertext is coming straight from the Client, we can use this as a comparison for the integrity check of the original file that was encrypted. This allows us to keep a fingerprint of the encrypted file, so if the original encrypted file loses integrity while in transit, the server will be able to compare the hashed ciphertext (copy of original) that was created client side, with another hashed ciphertext (copy of original) that was created on the server side. This will allow us to successfully test the

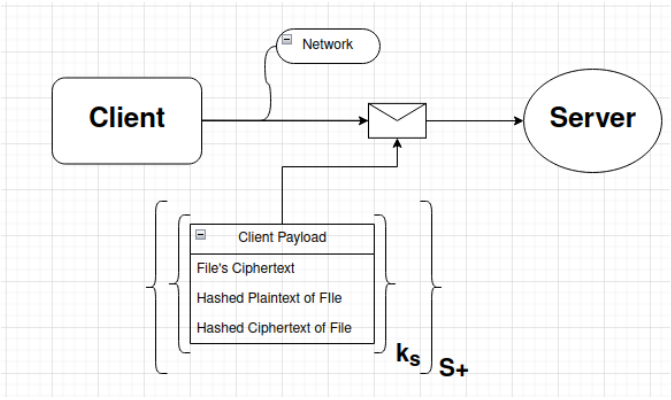
integrity of the original encrypted file that is to be stored on the server.

E. Client's Payload Before Being Sent Over Network to Server



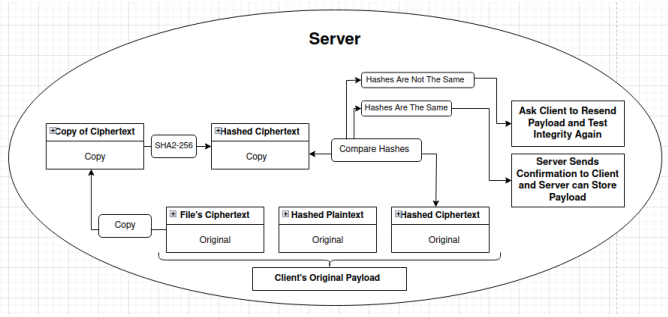
Here is what the final package of data will look like before we encrypt it using the session key and public key of the server.

F. Client Sending Files Over Network to Server With Proper Authentication



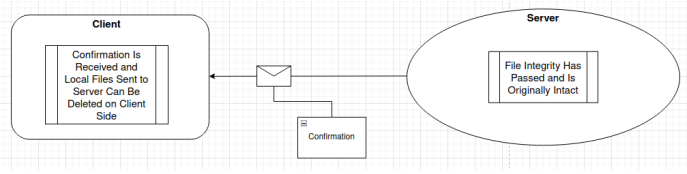
Here we can see the data in transit with the data encrypted with the session key and then the public key of the server. We encrypt the data with the session key that we established in the beginning of the communication of the client and server so that the data in transit is secure. This allows the server to know that the data being sent is from the client and not a reply or a man in the middle attack. The encryption with the public key of the server is for an additional layer of security that doesn't allow anyone else to see the data in transit if it were intercepted.

G. File Integrity Check When Client's Payload Arrives to Server



After the server uses its private key and the shared session key, it unpacks the encrypted client payload and temporarily stores the files from Client (File's Ciphertext, Hashed File's Plaintext, Hashed File Ciphertext). The server then needs to make a copy of the original ciphertext. That copy then needs to be hashed using SHA2-256 hashing algorithm in order to make a proper comparison to the hashed ciphertext that was originally sent by the Client. This comparison is the server's integrity check for the original encrypted file that is to be stored on the server. If the two hashed ciphertext files match exactly then the file integrity check has passed and the original encrypted file has kept its integrity throughout transit. The server is now able to send a confirmation message to the Client. Although, if the two hashed ciphertext files do not match exactly then the file integrity check has failed and the server will get rid of the payload it originally received from Client and will request the same files from the Client again. If file integrity continues to fail numerous times, then the Client will be notified of possible security issues/breaches.

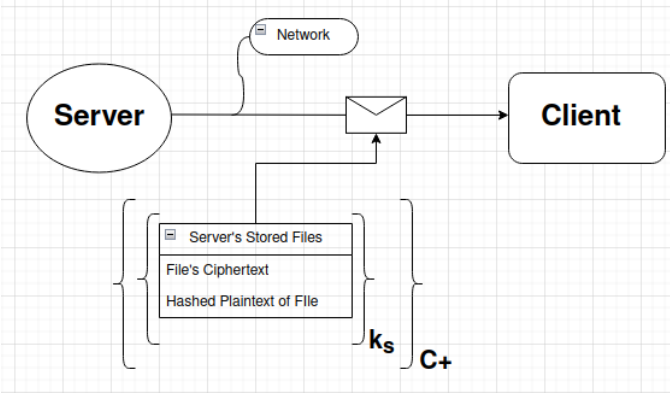
H. Server Sends Confirmation Signal to Client If File Integrity Check is Passed and Client is Able to Delete Local Files



If the server's file integrity check has passed then the integrity of the encrypted file has been kept throughout transit. Since integrity has been kept, the server can now store the encrypted file and hashed plaintext, originally sent by Client, for as long as it needs to. The server can now send the Client a confirmation message stating that the server indeed received those files and that they are intact and have not been tampered with. The Client can now safely delete those local files that had been sent to the server. Now the server is the only entity with those files. Since the server's integrity check passed, the server no longer needs the hashed ciphertext files, both

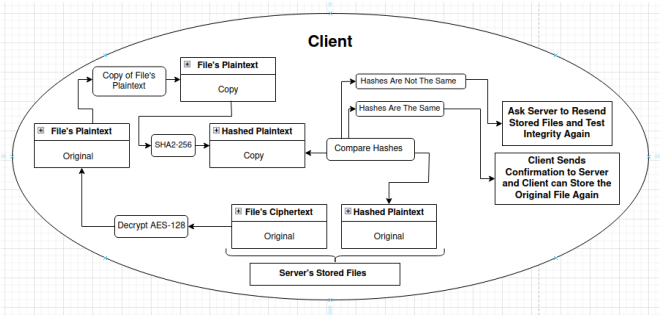
the original and the copy.

I. Server Sending Files Over Network to Client With Proper Authentication



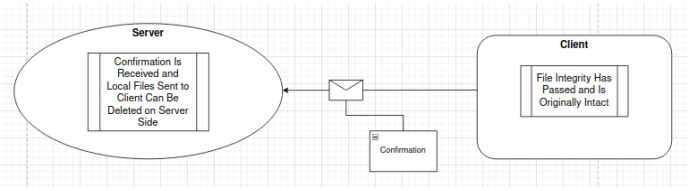
Here we can see the data stored on the server will be encrypted with the session key that was initiated when the client made the initial request for the files stored on the server. With this encrypted data, it will be encrypted again with the public key of the client so that only the client can view this data when in transit. We established the session key in the beginning of the communication of the client and server so that the data in transit is secure. This allows the client to know that the data being sent is from the server and not a reply or a man in the middle attack.

J. File Integrity Check When Server's Stored Files Arrives to Client



The server has now sent over its stored files to the Client, but the Client needs to make a file integrity check of the files (File's Ciphertext, File's Hashed Plaintext) that were sent over a network connection. In order to accomplish an integrity check, the Client must first decrypt the AES-128 encrypted file to receive the original plaintext. Once the plaintext is visible, the Client will now make a copy of said plaintext and hash that copy with SHA2-256. Now the Client has the original plaintext file, a copy of the plaintext that has been hashed, and the original the hashed plaintext that was sent over via server. The Client will now make a comparison between the hashed plaintext(copy) with the hashed plaintext(original). If the two hashed plaintext files are an exact match then the encrypted file, sent over by the server, has kept integrity throughout transit. The Client can now store the file's plaintext as a local file without any encryption of the file. The Client can now send a confirmation message to the server. Although, if the two hashed plaintext files do not match exactly then the file integrity check has failed and the Client will get rid of the payload it originally received from the server and will request the same files from the server again. If file integrity continues to fail numerous times, then the Client will be notified of possible security issues/breaches.

K. Client Confirms that receives files so server can delete stored files and session key



If the Client's file integrity check has passed then the integrity of the encrypted file has been kept throughout transit. Since integrity has been kept, the Client can now store the encrypted file as a plaintext file locally. The Client can now send the server a confirmation message stating that the Client indeed received those files and that they are intact and have not been tampered with throughout transit. The server can now safely delete those files that had been sent to the Client. Now the Client is the only entity with the plaintext file or any file(s). The transition of files from Client to server and server to Client is successful.

IV. IMPLEMENTATION

Before any implementation can be applied, there must be an active and live bluetooth connection between the client and the server. Without the initial connection via bluetooth, the program will fail to initiate any further. This peer-to-peer file sharing system was implemented to securely store and retrieve encrypted files between a client and a single server, that is represented as a raspberry pi. The system includes a client module and a server module. Each one handles its own encryption, its own transmission, the acknowledgement of incoming files. The client implements the functionality needed for file handling, encryption, decryption, and network/bluetooth communication. The client first does a hash of the plaintext file that is to be sent to the server. The client will create a hash result of the plaintext and write the hash value to a file that is dedicated to holding that hash value. We need this hash file throughout the entire cycle of the program inorder to do a file integrity check for when the

server sends files to the client. The client generates two types of AES-128 keys for two different encryption sequences. The first AES-128 key created is the static key; this static key is used for the initial encryption of the plaintext file and then is on a static value for the client to decrypt later on. Each time the AES-128 encryption or decryption function is called; it creates a new file with the appropriate name of whether the file contains ciphertext or decrypted ciphertext. The second AES-128 key created is our session key. The Diffie-Hellman algorithm is used to assist the creation of this key, along with a predefined salt value. Now that client has a static key for encrypting and decrypting files on the client side and a session key for simulating an encrypted packet being sent over a network medium. It is important to note that the session key is generated on both sides (client and server) and is the same key value. The client is now able to make one last encryption for the hash file. In the client's payload is an encrypted file, a hash file, and a file that hashes the encrypted file. Each file must be wrapped in another level of encryption, using AES-128 encryption and session key. It is also important to note, when the encryption function is called, the IV value for the AES-128 encryption is randomly generated each time it is called and writes that IV value to the beginning of the new encrypted file along with its other encrypted content. Each encrypted file carries its own randomly generated IV value at all times. Once all three files have been wrapped in the simulated encrypted network; each file will be sent one by one to the server. As soon as the server has received all three files and the bluetooth connection has stayed live, then the server will send an acknowledgement message to the client, stating the files have reached the server successfully. The server will then unpack the encrypted packet with the session key and store the decrypted files locally. Once the file integrity check has passed on the server; the server will send another acknowledgement to the client stating the client can now delete the files that were sent over the network medium. The user will then be prompted to decide whether they want the client to request the files from the server. If the user selects “N” then the files will remain stored on the server and the program will end and the bluetooth connection between the client and server. If the user inputs “Y” then the bluetooth connection will stay live and the client and the server will establish acknowledgments between each other for another file transfer. Server will re-encrypt the encrypted data file and the hash file to re-simulate the encrypted network connection. With both entities having the same session keys again the client will be able to unpack the encrypted package with ease. When the two files reach the client, the client will send an acknowledgement that it got the files successfully. Client will perform the file integrity check and if the integrity check returns true then the client can acknowledge the server that it is safe to delete the files stored on server. Client will clean up those useless files so that the only file left is the new_alice.txt file, which holds the same data as the original alice.txt file. The program has now made a successful cycle in storing files and retrieving them back.

V. Performance Testing

We decided to test the performance bluetooth capabilities between the client and the server. Specifically, we tested how far you could be before the relay of the files stopped the most or how far you could be before issues started to occur. Within a room by 25 feet x 25 feet; we found that there were no issues in the file transfer between devices. With the farthest corners of the room, we found that the speed of the files going through did not slow down but there were a few more instances where the live file transfer did abruptly stop. We found that after 30 feet to 40 feet the bluetooth connection slowed down considerably, to where the connection no longer had any successful file transfers between the two entities. As long as the client and the server were within 10 feet from each other; it took on average 5.3 seconds for the entire file transfer cycle to take place. We concluded that performance capabilities of the bluetooth connection were rather reliable and quick. It was only after two consecutive walls were used as the restricting obstacles, where the bluetooth was unable to connect at all and no data or packages were able to initiate to be sent.

VI. CONCLUSION

This Peer-to-Peer(P2P) file sharing system design allows a Client and a server to share files securely through the use of pycryptodome’s hashing and cryptographic algorithms. The implementation of the P2P file sharing system successfully accomplishes what is described and what we wanted the system to do and operate as.

REFERENCES

[1] “Welcome to pycryptodome’s documentation¶,” Welcome to PyCryptodome’s documentation - PyCryptodome 3.210b0 documentation, https://www.pycryptodome.org/ (accessed Mar. 25, 2024).